AUTOMATED WEB SCANNER

A Python-based audit of HTTP security headers and basic CORS behavior

Project Report

Prepared as an educational software project

Note: Use this tool only on systems you own or have explicit permission to test.

1. Abstract

The Automated Web Scanner is a lightweight Python command-line application designed to review selected HTTP security headers and perform a basic CORS behavior check. It sends ordinary HTTP requests, displays the response status and header results, and can export a JSON report for later review. It is an educational first-pass checker, not a replacement for a professional security assessment.

2. Problem Statement

Web applications use HTTP response headers and Cross-Origin Resource Sharing (CORS) rules to help control browser behavior and cross-origin access. Manually checking these settings can be repetitive. This project automates a small set of checks and presents the findings in a consistent format.

3. Objectives

• Check common security headers. • Send a non-invasive OPTIONS request with a synthetic Origin. • Report basic CORS response fields. • Save results in JSON when requested. • Encourage responsible, permission-based security testing.

4. Tools and Technologies

Programming language: Python 3.10 or newer. Library: Requests. Interface: command-line terminal. Output: readable text and optional JSON. The project uses HTTP GET and OPTIONS requests with a configurable timeout.

5. Security Headers Checked

Strict-Transport-Security encourages HTTPS use; Content-Security-Policy can reduce some injection risks; X-Content-Type-Options helps prevent MIME sniffing; X-Frame-Options can help mitigate clickjacking; Referrer-Policy controls referrer information; Permissions-Policy controls selected browser features. The appropriate configuration depends on each application.

6. CORS Check

The scanner sends an OPTIONS request with a synthetic Origin value and records Access-Control-Allow-Origin and Access-Control-Allow-Credentials. A wildcard origin is not automatically unsafe because it may be appropriate for public resources. The tool reports observations and a review warning rather than declaring a site vulnerable.

7. System Workflow

1. User enters an authorized HTTP/HTTPS URL. 2. The program validates and normalizes the URL. 3. It sends a GET request and records the final URL, status, and response headers. 4. It checks whether selected headers are present. 5. It sends a basic OPTIONS request for CORS observations. 6. It prints the report and optionally saves JSON.

8. How to Run

Install Python, create a virtual environment, install dependencies using `pip install -r requirements.txt`, then run `python scanner.py https://example.com`. To save machine-readable output, add `--json report.json`. See README.md for Windows and macOS/Linux commands.

9. Limitations

The scanner checks only a limited set of headers and a single basic CORS request. Results can depend on redirects, endpoint behavior, authentication, and application design. Missing headers are not automatically vulnerabilities. The scanner does not crawl pages, exploit issues, bypass authentication, or provide a comprehensive penetration test.

10. Conclusion and Future Scope

This project demonstrates how Python can automate basic web configuration checks. Possible future improvements include HTML reports, multiple authorized target lists, configurable header policies, unit tests, and a graphical interface. Any extension should retain clear authorization and safe rate limits.

11. Responsible Use

Only test websites you own or have explicit written permission to assess. Do not use the tool to disrupt services, access private data, or bypass controls. Follow applicable laws, organizational rules, and website terms.